H/w level Classes.

DATE 07/10/26

Microarchitectures & Memory Ordering Mechanics.

1. Out of Order Execution.
- Used in most high end processors, modern systems to make used of cycles that would otherwise be wasted.

~ In a traditional execution when an instruction is fetched, if the operands are available to it is executed else the CPU waits or stalls and when the operands are fetched then the execution occurs and results are returned

DATE

Core A	Core B
X = 1	Y = 1
print (Y)	print (X)

Since the updated values of X & Y are stored in local buffer stores of A & B, both the instructions print 0

→ Invalidate queues - Now, when a core modifies something, it must notify all other cores that the state of element that they have is invalid now or stale, to avoid stalling the receiver core; these messages are placed in the invalidate queues.

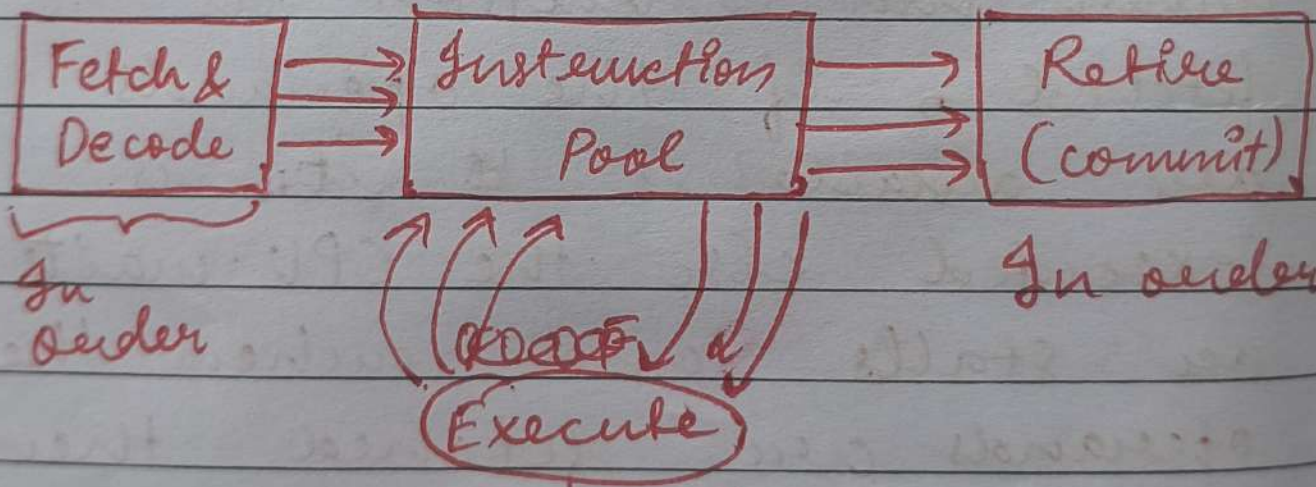
→ OOOE is done due to memory latency.
- Accessing RAM can take 100s of clock cycles.
- whereas executing an arithmetic op takes less than a single cycle.

So to keep CPUs busy we use.

1. Store buffers - The write or store instructions, data after execution is stored in a local FIFO queue called store buffer & then the core immediately moves to the next instruction.
* Without waiting for it to be written in cache line
global for cores.

DATE / /

~ But in OOOE if an instruction is too big to fetch, the CPU starts processing other instruction while the operands are being fetched and later it is executed. At the end the results are rearranged and returned in the original order.



Again since the cores shouldn't actually ~~also~~ applied it to cache yet they perform op on invalid or stale elements.

Hence they trade global memory consistency across cores for maximum single-threaded execution speed.

3. Speculative execution

To keep processing pipelines filled, modern processors don't wait to read an "if branch" Instead the branch predictor guesses and executes an op.

If correct → time saved on avoiding Conditional bran

- prohibits critical HW optimization.

- SB cannot delay writes.
- ID cannot delay processing invalidation message.
- Pipelines cannot reorder loads ahead of stores.

3. Cache Coherency & Inter-connected traffic.

- Modern CPUs process memory in 64-byte chunks called Cache Lines

→ Consitent Cache protocol :-
MESI (Modified, Exclusive, Shared, Invalid)

DATE ___/___/___

Memory Barriers & Memory Models.

1. load fence (l fence) - drains the local invalidate queue.

2. sfence (store fence) - drains the local store buffer.

3. mfence (memory f) - acts as a full h/w barrier, combines both

In Java :-

sfence - memory_order_release

lfence - memory order - aquire

2. Sequential Consistency

- Program Order
- Global Interleaving

The threads take turn in ordered sequence

DATE ___/___/___

if wrong :- It rollbacks and executes correct path but.

- Speculative load reordering :- since the data is received by cache, another core can read a value long before it is validated.

- Side-Channel Vulnerabilities - many malicious threads may deduce data they shouldn't access due to the speculative window

→ Store Buffers	Hides write latency.	Delayed visibility of executions.
→ Invalidate Queues	Prevents bus stalls.	o/p on stale values.
→ Speculative execution	Avoids pipeline stalls	loads execute ahead of

changes address if it holds an expected value.

Producer Thread

1. Writes payload to buffer_[head]

↓

Release fence / S Fence

Consumer Thread.



updated.

3. Acquire fence / L fence

↓

4. Reads 'buffer_[tail]' safely.

DATE ___/___/___

4. Lockfree Scale & Bypassing Context Switches.

Teraditionally OS locks like mutexes
But context switch wastes thousands of CPU cycles.

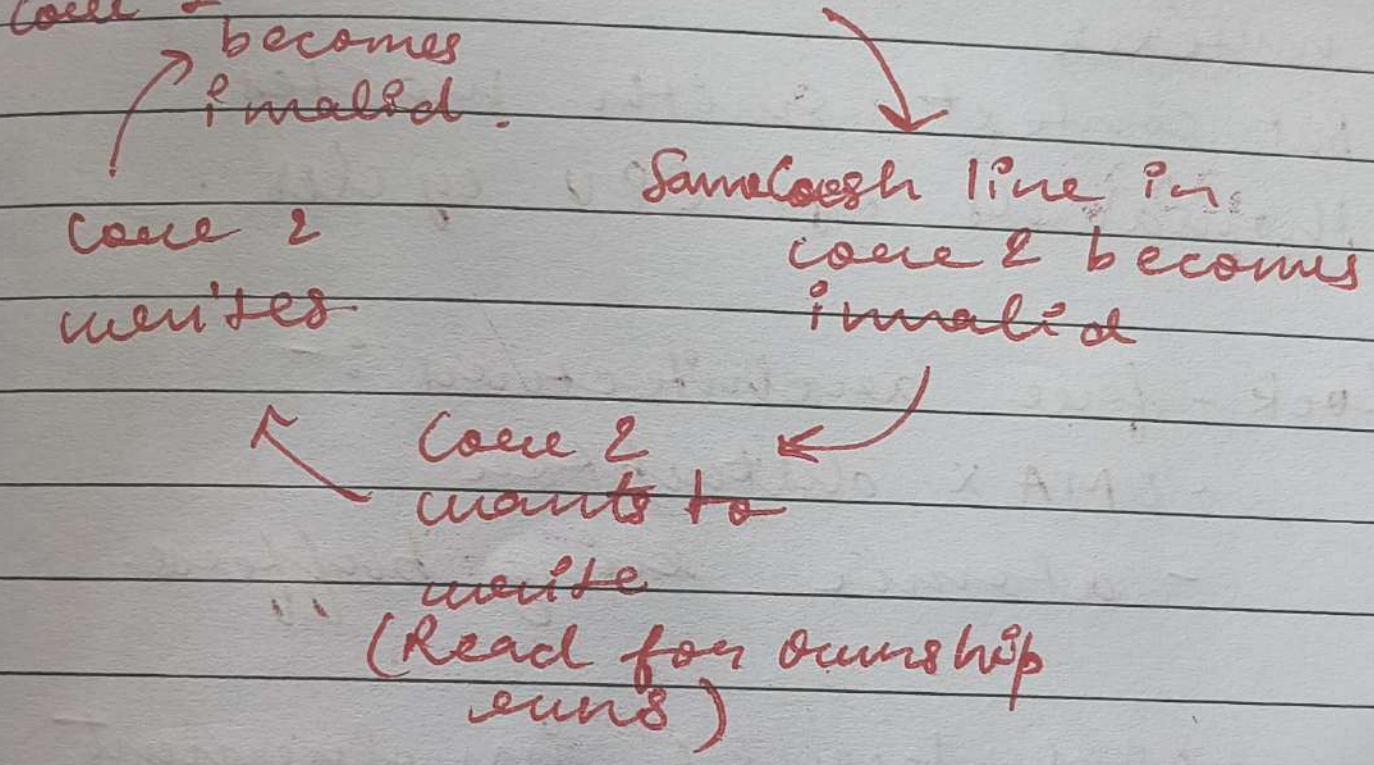
Lock-free architectures :-
- LMAX disruptor
- atomic ring buffers.

avoid kernel involvement using 2 techniques :-

1. Cache Padding - Adds bytes around variables so it has its own isolated 64 byte cache line
2. Atomic CAS (compare & swap) (lock cmpxchg) instruction

DATE / /

False Sharing → when 2 independent o/p lie in the same 64 byte cache line.



Vicious cycle.